Nombres:	**Matrículas:**
Adolfo Khaled Mendoza Cervantes	2169793
Eros Vargas Mariscal	2226517
Jesús Enrique García Mendoza	2226606
Kevin Armando Vargas Briones	2223097
Nombre del curso: Componentes de lógica de negocios	**Nombre del profesor:** Miguel Ángel Gómez Marroquín
Modulo Netacad: 5	**Actividad:** Patrones y SOLID
Fecha: 05/11/2026	

Procedimiento:

Patrones de diseño y principios SOLID

Este documento indica qué patrón se usó, dónde está en el código y por qué se eligió, y muestra un ejemplo real del repositorio para cada principio SOLID La ubicación de cada elemento puede seguirse en el diagrama de clases (Figura 1) y en el mapa de dominio (Figura 2) del anexo.

Resumen de patrones

#	Patrón	Dónde (principal)	Para qué
P-01	**Aggregate**	src/domain/*.py (7 Aggregate Roots)	Proteger invariantes INV-01 a INV-25
P-02	**Repository**	domain/repositorios.py · infrastructure/sql.py	Aislar el dominio de la base de datos
P-03	**Unit of Work**	application/puertos.py · SqlUnitOfWork	Guardar todo o nada por caso de uso
P-04	**Specification**	domain/especificaciones.py	Reglas combinables con AND, OR y NOT
P-05	**State**	domain/solicitud.py (TRANSICIONES)	Controlar el ciclo de vida de la solicitud
P-06	**Strategy**	domain/valores.py CriterioDeCertificacion	Criterio Regular u Oficial intercambiable
P-07	**CQRS**	application/consultas.py · *SqlReader	Separar escritura de lecturas públicas
P-08	**Adapter**	infrastructure/adapters.py	Conectar sistemas externos por puertos

1. Patrones de diseño

P-01: Aggregate

Campo	Contenido
Patrón	Aggregate
Dónde está en el código	src/domain: siete Aggregate Roots: SolicitudDeInscripcion (solicitud.py), Curso (curso.py), Grupo (grupo.py), Inscripcion (inscripcion.py), ExpedienteAcademico (expediente.py), Acta (acta.py) y Certificado (certificado.py). Estudiante (estudiante.py) es una Entity que se referencia por id. Grupo contiene la entidad interna RegistroEnListaDeEspera (composición; rombo en el diagrama de clases).
Cómo se aplica	Cada Aggregate es la única puerta para cambiar su estado y protege sus propias invariantes. Grupo.reservarLugar() valida INV-03 (los lugares ocupados no superan Cupo.capacidad) y lanza SinCupo; la lista de espera solo cambia con agregarAListaDeEspera() y liberarLugar(), que además ofrece el lugar al siguiente por 24 horas (INV-04). Los Aggregates se referencian entre sí por id (Inscripcion.grupoId, Grupo.ocupados guarda inscripcionId), nunca por objeto, y cada transición devuelve o registra su Domain Event (LugarReservado, InscripcionFormalizada, ActaCerrada, etc.).
Por qué se usó	Las invariantes del BRD (INV-01 a INV-25) necesitan un límite de consistencia claro: si otra clase modificara directamente grupo.ocupados, el cupo podría romperse. Referenciar por id mantiene los Aggregates pequeños, permite guardar cada uno con su Repository y limita una transacción a los Aggregates que toca el caso de uso (ADR-001).
Trazabilidad	tests/unit/test_dominio_u3.py, tests/unit/test_reglas_u5.py y tests/unit/test_solicitud.py. Diagrama de clases, capa Dominio (estereotipo «Aggregate Root»); mapa de dominio (recuadros «Aggregate»).

Fragmento del código

```
# src/domain/grupo.py
def reservarLugar(self, inscripcionId: str) -> LugarReservado:
    # INV-03: los lugares ocupados no superan la capacidad.
    if len(self.ocupados) >= self.cupo.capacidad:
        raise SinCupo("El grupo alcanzó su cupo máximo")
    self.ocupados.append(inscripcionId)
    return LugarReservado(self.grupoId, inscripcionId, self.fechaInicio + timedelta(days=7))
```

P-02: Repository

Campo	Contenido
Patrón	Repository
Dónde está en el código	Interfaces: src/domain/repositorios.py, ocho clases abstractas (ABC): CursoRepository, GrupoRepository, InscripcionRepository, ExpedienteRepository, ActaRepository, CertificadoRepository, EstudianteRepository y SolicitudRepository. Implementaciones: src/infrastructure/sql.py, SqlCursoRepository, SqlGrupoRepository, SqlInscripcionRepository, etc., sobre la clase base _SqlBase.
Cómo se aplica	Las interfaces hablan en el lenguaje del dominio (obtener, guardar, agregar, todos, delGrupo, siguienteId) y no exponen SQL. _SqlBase concentra el select, insert y update con SQLAlchemy Core; cada Aggregate se guarda serializado en la columna datos, con columnas aparte para consultas (estado, saldo, folio, total). Cuando no existe el registro, obtener() lanza NoEncontrado, que la API traduce a 404.
Por qué se usó	El dominio no debe depender de la base de datos. Con la interfaz en el dominio y la implementación en infraestructura, el mismo código corre sobre PostgreSQL (Docker y Render) y sobre SQLite (pruebas) cambiando solo DATABASE_URL, y los servicios de aplicación no cambian (ADR-006).
Trazabilidad	ADR-006 (persistencia). tests/integration/test_solicitud_service.py::test_postgresql_formaliza_y_revierte. Diagrama de clases: «interface» 8 Repository en Dominio y «Adapter · 8 clases» Sql*Repository en Infraestructura.

Fragmento del código

```
# src/domain/repositorios.py (interfaz del dominio)
class GrupoRepository(ABC):
    @abstractmethod
    def obtener(self, grupoId: str) -> Grupo: ...
    @abstractmethod
    def guardar(self, grupo: Grupo) -> None: ...

# src/infrastructure/sql.py (implementación)
class SqlGrupoRepository(_SqlBase, GrupoRepository):
    tabla = grupos
    def obtener(self, grupoId): return self._obtener(grupoId)
    def guardar(self, grupo): self._guardar(grupo.grupoId, grupo)
```

P-03: Unit of Work

Campo	Contenido
Patrón	Unit of Work
Dónde está en el código	Interfaz UnitOfWork y administrador de contexto transaccion(uow) en src/application/puertos.py. Implementación SqlUnitOfWork en src/infrastructure/sql.py (BEGIN / COMMIT / ROLLBACK sobre la conexión compartida de BaseDeDatos). Uso: todos los Application Services abren su caso de uso con: with transaccion(self.uow).
Cómo se aplica	transaccion() llama a iniciar(); si el bloque termina bien llama a confirmar() y, si se lanza cualquier excepción, llama a cancelar() y vuelve a lanzarla. Ejemplo: SolicitudService.formalizar() cambia la solicitud a Inscrito y reserva lugares en varios grupos; si un grupo lanza SinCupo a mitad del ciclo, se revierten la solicitud y los grupos ya modificados. En InscripcionService.liberarPorPlazoVencido() la notificación se envía después de confirmar, para no avisar de un lugar que no se liberó.
Por qué se usó	Un caso de uso modifica varios Aggregates a la vez (Inscripción, Grupo y Expediente al inscribirse); deben guardarse todos o ninguno. Concentrar la transacción en un solo objeto evita que cada Repository haga su propio commit (ADR-002 de la U3 y ADR-006).
Trazabilidad	tests/integration/test_integracion_u5.py::test_PIA14_rollback_datos_incompletos_no_guardan_nada y ::test_PIA15_unidad_de_trabajo_rollback_directo; tests/integration/test_solicitud_service.py::test_sin_cupo_revierte_toda_la_transaccion. Diagrama de clases: «interface» UnitOfWork y «Adapter» SqlUnitOfWork.

Fragmento del código

```
# src/application/puertos.py
@contextmanager
def transaccion(uow: UnitOfWork):
    """Inicia, confirma o cancela: si algo falla no se guarda nada."""
    uow.iniciar()
    try:
        yield
    except Exception:
        uow.cancelar()
        raise
    uow.confirmar()
```

P-04: Specification

Campo	Contenido
Patrón	Specification
Dónde está en el código	src/domain/especificaciones.py: clase base Specification (es_satisfecha_por, incumplidas, operadores &, |, ~), compuestos Conjuncion, Disyuncion y Negacion, y 10 reglas hoja (EstudianteActivo, TieneLineas, Autorizada, EstudiantePreferente, TotalMinimo, PromocionActiva, CalificacionMinima, AsistenciaMinima, ConexionMinima y GrabacionVistaATiempo). Reglas compuestas con nombre: PUEDE_PROCESARSE, BENEFICIO_REFERENCIA, ASISTENCIA_EN_LINEA y CriterioDeCertificacion.regla() (src/domain/valores.py).
Cómo se aplica	Cada regla del negocio es un objeto que responde sí o no sin modificar nada, y se combina con AND (&), OR (|) y NOT (~). incumplidas() devuelve el nombre de la condición que falló, y ese texto llega al usuario en el mensaje de error. Lo usan SolicitudDeInscripcion.formalizar(), SolicitudService.evaluar(), AsistenciaRegistrada.acreditaEnLinea() (llamado por LmsAdapter) y CriterioDeCertificacion.evaluar() (llamado por ConsolidadorDeActa).
Por qué se usó	Varias reglas del BRD y del paquete de datos del PIA son condiciones combinadas: INV-12 es un OR (60 % de conexión o grabación vista en 72 horas), INV-21 es un AND (calificación y asistencia mínimas) y el beneficio por referencia incluye un NOT (sin promoción activa). Con Specification la regla se lee igual que en el BRD, cada condición se prueba sola y el rechazo dice el motivo concreto en lugar de repetir if en cada método (ADR-004).
Trazabilidad	ADR-004. tests/unit/test_especificaciones.py (test_and_or_not_crean_reglas_compuestas, test_puede_procesarse_dice_que_falta, test_INV12_asistencia_en_linea_es_un_OR, test_INV21_criterio_regular_es_un_AND). Diagrama de clases: recuadro Specification Pattern (ADR-004); mapa de dominio: ASISTENCIA_EN_LINEA y Criterio.regla().

Fragmento del código

```
# src/domain/especificaciones.py
PUEDE_PROCESARSE = EstudianteActivo() & TieneLineas() & Autorizada()
BENEFICIO_REFERENCIA = EstudiantePreferente() & TotalMinimo(Decimal("1500")) & ~PromocionActiva()
ASISTENCIA_EN_LINEA = ConexionMinima() | GrabacionVistaATiempo()      # INV-12

# src/domain/solicitud.py (formalizar)
incumplidas = PUEDE_PROCESARSE.incumplidas(CandidatoSolicitud(self, estudiante))
if incumplidas:
    raise SolicitudNoProcesable("No se puede formalizar: falta " + ", ".join(incumplidas))
```

P-05: State

Campo	Contenido
Patrón	State
Dónde está en el código	src/domain/solicitud.py: enum EstadoSolicitud (Preinscrito, Inscrito, Concluido, Liberada), tabla TRANSICIONES y método _exigir() del Aggregate SolicitudDeInscripcion. Mismo enfoque, más simple, en Inscripcion (EstadoInscripcion: Preinscrito → Inscrito → Liberada) y en Acta (EstadoActa: Abierta → Cerrada).
Cómo se aplica	La tabla TRANSICIONES asocia cada acción con el estado que requiere y el estado destino (formalizar: Preinscrito → Inscrito; concluir: Inscrito → Concluido; cancelar: Preinscrito → Liberada). Cada transición sigue tres pasos: valida el estado con _exigir(), cambia el estado y registra el Domain Event. Si el estado no es el requerido lanza TransicionNoPermitida (409 en la API). La propiedad accionesPermitidas se calcula de la misma tabla y la API la devuelve, así el cliente sabe qué puede hacer en cada estado.
Por qué se usó	El ciclo de vida limita qué operaciones son válidas: sin control de estado se podría concluir una solicitud cancelada o formalizarla dos veces (y generar dos folios, INV-08). Se eligió la variante con tabla de transiciones en lugar de una clase por estado porque los estados no tienen comportamiento propio aparte de qué acciones aceptan; la tabla es más corta y se lee como el diagrama de estados.
Trazabilidad	tests/unit/test_solicitud.py::test_estados_finales_no_aceptan_cambios; tests/integration/test_paquete_profesor.py::test_transiciones_del_paquete; tests/integration/test_api.py::test_409_transicion_no_permitida. Mapa de dominio: recuadros punteados «Estado» de Solicitud, Inscripción y Acta.

Fragmento del código

```
# src/domain/solicitud.py
TRANSICIONES = {
    "formalizar": (EstadoSolicitud.PREINSCRITO, EstadoSolicitud.INSCRITO),
    "concluir":   (EstadoSolicitud.INSCRITO, EstadoSolicitud.CONCLUIDO),
    "cancelar":   (EstadoSolicitud.PREINSCRITO, EstadoSolicitud.LIBERADA),
}
def _exigir(self, accion, requerido):
    if self.estado is not requerido:
        raise TransicionNoPermitida(f"No se puede {accion} una solicitud en estado {self.estado.value}")
```

P-06: Strategy

Campo	Contenido
Patrón	Strategy
Dónde está en el código	Value Object CriterioDeCertificacion en src/domain/valores.py, con sus dos variantes regular() (70 de calificación y 80 % de asistencia) y oficial() (80 y 90 %). Cada Curso guarda su estrategia en el atributo criterio (src/domain/curso.py); la usa el Domain Service ConsolidadorDeActa.consolidar() (src/domain/servicios.py). Ponderacion (40/40/20) se inyecta de la misma forma al calcular la Calificación Final.
Cómo se aplica	ConsolidadorDeActa recibe el criterio del curso y llama criterio.evaluar(calificacion, asistencia) sin saber si es Regular u Oficial; la decisión de cuál aplicar se toma al crear el Curso. Cada estrategia arma su regla con Specification (CalificacionMinima & AsistenciaMinima) y devuelve si cumple y el motivo si no cumple.
Por qué se usó	El BRD define dos criterios de certificación intercambiables por curso (INV-02 e INV-21). Con Strategy el algoritmo de consolidación del acta es el mismo para todos los cursos y agregar otro criterio no obliga a modificar ConsolidadorDeActa ni ActaService (ADR-002 de la U3, complementado por ADR-004).
Trazabilidad	tests/unit/test_especificaciones.py::test_INV21_criterio_regular_es_un_AND y ::test_INV21_criterio_oficial. Diagrama de clases: «Value Object» CriterioDeCertificacion y «Domain Service» ConsolidadorDeActa; mapa de dominio: Criterio de Certificación (Regular 70 y 80 % · Oficial 80 y 90 %).

Fragmento del código

```
# src/domain/valores.py
@staticmethod
def regular() -> "CriterioDeCertificacion": return CriterioDeCertificacion(70, 80)
@staticmethod
def oficial() -> "CriterioDeCertificacion": return CriterioDeCertificacion(80, 90)

# src/domain/servicios.py (ConsolidadorDeActa.consolidar)
cumple, motivo = criterio.evaluar(calificacion, asistencia)       # INV-21
```

P-07: CQRS

Campo	Contenido
Patrón	CQRS
Dónde está en el código	Commands (escritura): SolicitarInscripcion en src/application/inscripcion_service.py y RegistrarSolicitud en src/application/solicitud_service.py, procesados por los Application Services con Aggregates y Unit of Work. Queries (lectura): VerificacionPublicaService y EstadoDePagoQuery en src/application/consultas.py, que usan las interfaces VerificacionPublicaReader y EstadoDePagoReader y los DTO de src/application/puertos.py. Implementaciones VerificacionPublicaSqlReader y EstadoDePagoSqlReader en src/infrastructure/sql.py, y tabla de proyección verificacion_publica.
Cómo se aplica	Las escrituras cargan el Aggregate, aplican sus reglas y lo guardan dentro de una transacción. Las consultas no cargan Aggregates: leen solo las columnas autorizadas y devuelven un DTO inmutable. Al emitir un certificado, SqlCertificadoRepository.agregar() actualiza la proyección verificacion_publica (titular, curso, fecha de conclusión y vigencia) en la misma transacción. EstadoDePagoSqlReader lee estado, saldo y fecha límite de las columnas de la tabla inscripciones.
Por qué se usó	La verificación pública (CU-05) no tiene autenticación y solo puede mostrar ciertos datos (sin contacto ni calificaciones, US-19); con un modelo de lectura separado es imposible exponer de más por error. Además leer sin deserializar el Aggregate es más simple y rápido. Es un CQRS ligero: misma base de datos, modelos de escritura y lectura separados (ADR-003).
Trazabilidad	ADR-003. Endpoints GET /verificacion/{folio} y GET /inscripciones/{inscripcionId}/estado-de-pago en src/api/main.py. Diagrama de clases: «Query», «interface · modelo de lectura» y «Adapter · modelo de lectura»; mapa de dominio: «Modelo de lectura» Verificación Pública y Estado de Pago.

Fragmento del código

```
# src/application/consultas.py
class VerificacionPublicaService:
    def __init__(self, reader: VerificacionPublicaReader):
        self.reader = reader
    def consultarCertificado(self, folioOQr: str) -> VerificacionPublicaDto:
        folio = folioOQr.rstrip("/").split("/")[-1]   # el QR lleva una URL que termina en el folio
        dto = self.reader.consultar(folio)
```

P-08: Adapter

Campo	Contenido
Patrón	Adapter
Dónde está en el código	src/infrastructure/adapters.py: NotificacionGatewayAdapter (implementa NotificacionGateway), ControlDeAccesoAdapter (implementa ControlDeAcceso), SistemaContableAdapter y LmsAdapter. Los puertos que se adaptan están en src/application/puertos.py; las clases Sql* de src/infrastructure/sql.py también son adaptadores hacia la base de datos.
Cómo se aplica	SistemaContableAdapter.pagoNotificado() traduce el aviso de pago del sistema contable externo a InscripcionService.acreditarPago(). LmsAdapter.conexionReportada() recibe el porcentaje de conexión y las horas hasta ver la grabación, los convierte en presente o ausente con ASISTENCIA_EN_LINEA y llama a AsistenciaService.registrarAsistencia() con el usuario sistema-lms. ControlDeAccesoAdapter resuelve los roles que en producción vendrían del proveedor de identidad (D-06).
Por qué se usó	Los sistemas externos (contabilidad, LMS o videoconferencia, correo y autenticación) tienen su propia interfaz, pueden cambiar o no existir en la demostración. El núcleo solo conoce los puertos y los adaptadores traducen; así se reemplaza un sistema externo sin tocar el dominio ni los servicios, y en las pruebas se usa Mock(spec=NotificacionGateway).
Trazabilidad	tests/integration/test_integracion_u5.py::test_pago_notificado_por_sistema_contable y las pruebas de lista de espera con Mock(spec=NotificacionGateway). Diagrama de clases: «Adapter · sistemas externos»; mapa de dominio: Sistema contable (externo) y LMS / videoconferencia (externo).

Fragmento del código

```
# src/infrastructure/adapters.py
class SistemaContableAdapter:
    """El sistema contable notifica pagos acreditados (supuesto del BRD)."""
    def __init__(self, servicio: InscripcionService):
        self.servicio = servicio
    def pagoNotificado(self, inscripcionId: str, monto: float, fecha: date):
        return self.servicio.acreditarPago(inscripcionId, monto, fecha)
```

2. Principios SOLID

Para cada principio se muestra un ejemplo real del código del proyecto.

S: Responsabilidad única (SRP)

Campo	Contenido
Principio	Responsabilidad única (SRP)
Ejemplo real	Cierre de acta (CU-04) repartido en cuatro clases, cada una con una sola razón para cambiar.
Dónde está en el código	src/application/acta_service.py (ActaService), src/domain/servicios.py (ConsolidadorDeActa y CalendarioDeDiasHabiles) y src/domain/acta.py (Acta).
Explicación	ActaService solo orquesta: obtiene los datos, abre la transacción y guarda. ConsolidadorDeActa solo arma las líneas del acta (calificación final, porcentaje de asistencia, cumple o no). CalendarioDeDiasHabiles solo calcula plazos en días hábiles. Acta solo valida si puede cerrarse (INV-18, INV-19, INV-22). Si cambia la regla de días inhábiles se toca solo el calendario; si cambia la forma de consolidar, solo el consolidador.

Fragmento del código

```
# src/application/acta_service.py (cerrarActa)
lineas, sinAsistencia = self.consolidador.consolidar(expedientes, curso.criterio, curso.ponderacion,
                                                     len(grupo.sesiones))
permitida = self.calendario.sumarDiasHabiles(grupo.ultimaSesion, 5)      # INV-18
acta.cerrar(lineas, sinAsistencia, hoy, permitida, grupo.ultimaSesion)   # INV-19, INV-22
```

O: Abierto/cerrado (OCP)

Campo	Contenido
Principio	Abierto/cerrado (OCP)
Ejemplo real	Agregar reglas de negocio nuevas sin modificar las existentes con el Specification Pattern.
Dónde está en el código	src/domain/especificaciones.py.
Explicación	Para incorporar las reglas del paquete de datos del PIA (BENEFICIO_REFERENCIA) se crearon subclases nuevas (EstudiantePreferente, TotalMinimo, PromocionActiva) y se combinaron con &, | y ~; no se modificó la clase Specification, ni Conjuncion, Disyuncion o Negacion, ni las reglas que ya usaba PUEDE_PROCESARSE. El código queda abierto a extensión (una clase más) y cerrado a modificación.

Fragmento del código

```
class TotalMinimo(Specification):                # extensión: una clase nueva
    def __init__(self, minimo: Decimal):
        self.minimo = minimo
        self.nombre = f"total >= {minimo} MXN"
    def es_satisfecha_por(self, c: CandidatoSolicitud) -> bool:
        return c.solicitud.total.monto >= self.minimo

BENEFICIO_REFERENCIA = EstudiantePreferente() & TotalMinimo(Decimal("1500")) & ~PromocionActiva()
```

L: Sustitución de Liskov (LSP)

Campo	Contenido
Principio	Sustitución de Liskov (LSP)
Ejemplo real	Cualquier implementación de un puerto o Repository sustituye a otra sin que el servicio note la diferencia.
Dónde está en el código	src/application/inscripcion_service.py (InscripcionService), src/infrastructure/adapters.py (NotificacionGatewayAdapter), src/infrastructure/sql.py (Sql*Repository) y tests/integration/test_integracion_u5.py.
Explicación	InscripcionService recibe un NotificacionGateway: en producción es NotificacionGatewayAdapter y en las pruebas es Mock(spec=NotificacionGateway), y el servicio funciona igual con ambos. Del mismo modo, los Sql*Repository cumplen el contrato de su interfaz con SQLite o con PostgreSQL (mismo comportamiento, obtener() lanza NoEncontrado en ambos casos), por eso las mismas pruebas corren en los dos motores.

Fragmento del código

```
# tests/integration/test_integracion_u5.py
aviso = Mock(spec=NotificacionGateway)       # no se envía ningún correo real

# src/infrastructure/adapters.py
class NotificacionGatewayAdapter(NotificacionGateway):
    def notificarLugarDisponible(self, estudianteId, vence):
        self.enviadas.append((estudianteId, vence))
```

I: Segregación de interfaces (ISP)

Campo	Contenido
Principio	Segregación de interfaces (ISP)
Ejemplo real	Interfaces pequeñas: una por Aggregate o por necesidad, sin métodos que el cliente no use.
Dónde está en el código	src/domain/repositorios.py y src/application/puertos.py.
Explicación	En vez de un repositorio genérico con todos los métodos, cada interfaz tiene solo lo que su cliente necesita: CertificadoRepository solo agregar() (un certificado no se modifica), ExpedienteRepository obtener() y guardar(), ControlDeAcceso solo tieneRol() y NotificacionGateway solo notificarLugarDisponible(). Los modelos de lectura (VerificacionPublicaReader, EstadoDePagoReader) están separados de los Repository, así CertificadoService no depende de consultas y VerificacionPublicaService no puede escribir.

Fragmento del código

```
# src/domain/repositorios.py
class CertificadoRepository(ABC):
    @abstractmethod
    def agregar(self, certificado: Certificado) -> None: ...

# src/application/puertos.py
class ControlDeAcceso(ABC):
    @abstractmethod
    def tieneRol(self, usuarioId: str, rol: str) -> bool: ...
```

D: Inversión de dependencias (DIP)

Campo	Contenido
Principio	Inversión de dependencias (DIP)
Ejemplo real	Los servicios dependen de abstracciones y la composición concreta se hace en un solo lugar.
Dónde está en el código	src/sistema.py (composition root) y los constructores de los Application Services, por ejemplo src/application/inscripcion_service.py.
Explicación	InscripcionService declara en su constructor UnitOfWork, CursoRepository, GrupoRepository, InscripcionRepository, ExpedienteRepository y NotificacionGateway, todas interfaces. La clase Sistema es el único lugar que conoce las clases concretas (SqlUnitOfWork, Sql*Repository, adapters) y las inyecta. La dependencia va hacia adentro: src/domain no importa SQLAlchemy, FastAPI, application ni infrastructure, y la infraestructura implementa las interfaces que define el núcleo.

Fragmento del código

```
# src/application/inscripcion_service.py
def __init__(self, uow: UnitOfWork, cursos: CursoRepository, grupos: GrupoRepository,
             inscripciones: InscripcionRepository, expedientes: ExpedienteRepository,
             notificaciones: NotificacionGateway):

# src/sistema.py
uow = SqlUnitOfWork(db)
self.inscripcion = InscripcionService(uow, self.cursos, self.grupos, self.inscripciones,
                                       self.expedientes, self.notificaciones)
```

Anexo: diagramas

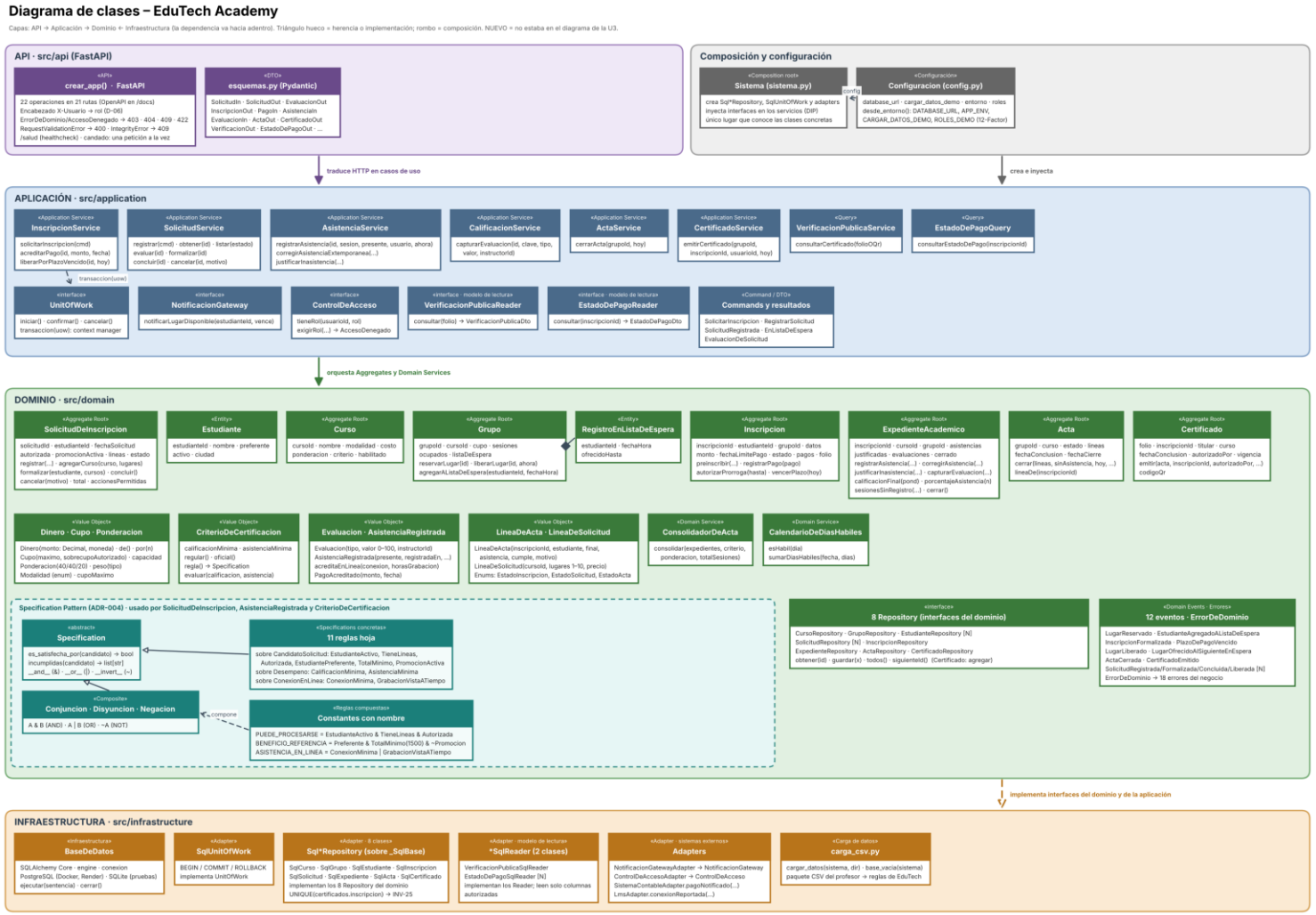


Figura 1. Diagrama de clases de EduTech Academy : capas API, Aplicación, Dominio e Infraestructura.

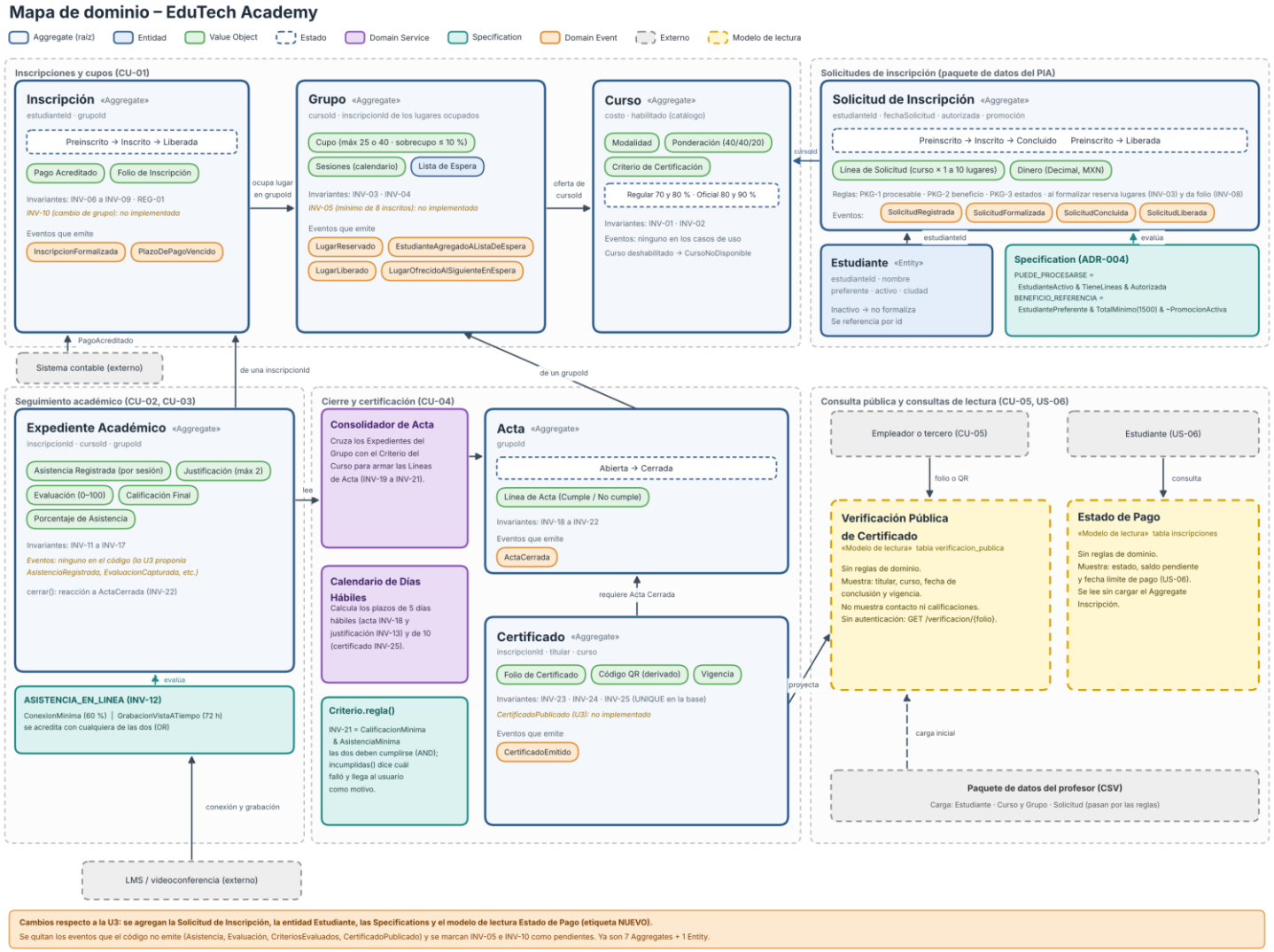


Figura 2. Mapa de dominio de EduTech Academy): Aggregates, Value Objects, estados, Specifications y modelos de lectura.